Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: ” Informații invizibile: forensics și steganografie”

A efectuat : st. gr.SI – 263 Morozovschii Carolina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Capitolul 1. Text și octeți: încălzirea (10 min)

Creează arsenal/forensics/decode.py. Fixează-ți întâi diferența dintre text și octeți, de care atârnă tot.

```
s = "Salut"
b = s.encode()          # bytes
print(b, b[0])          # b"Salut" 83
# TODO: tipareste b.hex() si refa bytes din hex
#       cu bytes.fromhex(...)
```

De observat. `b[0]` e un număr, nu o literă. Tot ce urmează lucrează pe `bytes`, nu pe text.

```
import base64
import re
from urllib.parse import unquote
import json
import argparse


# A1: textul poate fi transformat în octeți și înapoi în hex
text = "Salut"
octeti = text.encode("utf-8")
print("Octeți:", octeti)
print("Primul octet:", octeti[0])
print("Hex:", octeti.hex())
print("Refăcut din hex:", bytes.fromhex(octeti.hex()))
```

Fig.1.1-Codul sursă (decode.py)

```
(.venv) PS C:\Users\moroz\OneDrive\Documents\laborator6> .\.venv\Scripts\python.exe arsenal\forensics\decode.py probe\mesaj.txt --fisier
Octeți: b'Salut'
Primul octet: 83
Hex: 53616c7574
Refăcut din hex: b'Salut'
```

Fig.1.2-Rezultatul codului (decode.py)

Capitolul 2. Recunoști stratul după formă (14 min)

Scrie o funcție care, dat un șir, ghicește ce codare e, uitându-se la caractere. Nu decoda încă, doar clasifică.

```
import re
def ghici_strat(s):
    # TODO: daca are %xx -> "url"
    # TODO: daca doar 0-9 a-f si lungime para -> "hex"
    # TODO: daca doar A-Za-z0-9+/= -> "base64"
    return "necunoscut"
print(ghici_strat("%2Fetc"))   # url
print(ghici_strat("48656c6c6f"))  # hex
```

De observat. Ordinea contează: verifică întâi „%" (URL), apoi hexul (mai strict), apoi base64 (cel mai larg alfabet).

```python
# A2: ghicește tipul codării după caractere
def ghici_strat(s):
    if re.search(r"%[0-9a-fA-F]{2}", s):
        return "url"

    if len(s) > 0 and len(s) % 2 == 0 and re.fullmatch(r"[0-9a-fA-F]+", s):
        return "hex"

    if len(s) > 0 and re.fullmatch(r"[A-Za-z0-9+/]*={0,2}", s):
        try:
            base64.b64decode(s, validate=True)
            return "base64"
        except Exception:
            pass

    return "necunoscut"
```

Fig.2.1-Codul sursă (decode.py)

```
Test URL: url
Test hex: hex
```

Fig.2.2-Rezultatul codului (decode.py)

Capitolul 3. Desfaci stratul recunoscut (14 min)

Acum chiar decodează, în funcție de ce a zis A2. Întoarce și tipul, și rezultatul.

```python
import base64
from urllib.parse import unquote
def desfa(s):
    strat = ghici_strat(s)
    # TODO: pentru fiecare strat, decodeaza
    #   base64 -> base64.b64decode(s)
    #   hex    -> bytes.fromhex(s)
    #   url    -> unquote(s).encode()
    # intoarce (strat, rezultat_bytes)
    return strat, s.encode()
```

```python
# A3: decodează stratul identificat
def desfa(s):
    strat = ghici_strat(s)

    if strat == "base64":
        rezultat = base64.b64decode(s, validate=True)
    elif strat == "hex":
        rezultat = bytes.fromhex(s)
    elif strat == "url":
        rezultat = unquote(s).encode("utf-8")
    else:
        rezultat = s.encode("utf-8")

    return strat, rezultat


print("Test URL:", ghici_strat("%2Fetc"))
print("Test hex:", ghici_strat("48656c6c6f"))

strat, rezultat = desfa("U2FsdXQ=")
print("Test Base64:", strat, rezultat)
```

Fig.3.1-Codul sursă (decode.py)

```
Test Base64: base64 b'Salut'
```

Fig.3.2-Rezultatul codului (decode.py)

Capitolul 4. Straturi multiple, într-o buclă (14 min)

Un rezultat poate fi tot o codare. Decodează în buclă până iese text citibil (sau te oprești după câteva încercări, ca să nu intri în buclă infinită).

```
def citibil(b):
    # TODO: True daca toti octetii sunt 32..126
    return all(32 <= c < 127 for c in b)

val = "NTM1OTU3NTk="
for _ in range(8):
    strat, b = desfa(val)
    # TODO: daca citibil(b): opreste si tipareste
    #       altfel val = b.decode(); continua
```

```python
def citibil(b):
    return all(32 <= octet < 127 for octet in b)


def desfa_straturi(text):
    valoare = text.strip()

    for _ in range(8):
        strat, rezultat = desfa(valoare)

        if strat == "necunoscut":
            return rezultat

        # Dacă rezultatul e text imprimabil, verificăm dacă mai e codat.
        if citibil(rezultat):
            text_nou = rezultat.decode("ascii")
            if ghici_strat(text_nou) == "necunoscut":
                return rezultat
            valoare = text_nou
        else:
            try:
                valoare = rezultat.decode("ascii")
            except UnicodeDecodeError:
                return rezultat

    return rezultat


with open("probe/mesaj.txt", "r", encoding="utf-8") as fisier:
    mesaj_codat = fisier.read()

mesaj_clar = desfa_straturi(mesaj_codat)
print("Mesaj decodat:", mesaj_clar.decode("utf-8"))
```

Fig.4.1-Codul sursă (decode.py)

```
Mesaj decodat: FLAG{straturi_de_encoding}
```

Fig.4.2-Rezultatul codului (decode.py)

Capitolul 5. Spargi un XOR cu cheie de un octet (16 min)

Citește probe/xor.bin (octeți), încearcă toate cele 256 de chei și păstrează rezultatul care arată a text.

```
date = open("probe/xor.bin", "rb").read()
for k in range(256):
    clar = bytes(b ^ k for b in date)
    # TODO: daca citibil(clar):
    #       print("cheie:", k, clar.decode())
```

De observat. Operatorul ^ e XOR; `b ^ k` aplică cheia pe un octet. O cheie de un octet are fix 256 de valori, deci le încerci pe toate.

```python
from pathlib import Path


def sparge_xor(cale):
    date = Path(cale).read_bytes()

    for cheie in range(256):
        clar = bytes(octet ^ cheie for octet in date)

        if citibil(clar):
            print("Cheia XOR:", cheie, f"(0x{cheie:02X})")
            print("Mesaj:", clar.decode("ascii"))
            return

    print("Nu am găsit o cheie care să producă text imprimabil.")


sparge_xor("probe/xor.bin")
```

Fig.5.1-Codul sursă (decode.py)

```
Cheia XOR: 128 (0x80)
Mesaj: Un singur octet tine tot secretul... gaseste-l! FLAG{xor_un_octet}
```

Fig.5.2-Rezultatul codului (decode.py)

Capitolul 6. Spargi un hash prin dicționar (16 min)

Din probe/hashuri.txt ia un hash. Identifică algoritmul după lungime, apoi caută parola într-o wordlist.

```
import hashlib
tinta = open("probe/hashuri.txt").readline().strip()
print(len(tinta))   # 32 md5 / 40 sha1 / 64 sha256
for cuv in open("probe/wordlist.txt", encoding="latin1"):
    cuv = cuv.strip()
    # TODO: daca hashlib.md5(cuv.encode()).hexdigest()
    #       == tinta: print("gasit:", cuv); break
```

De observat. Nu „decriptezi" hash-ul: hashuiești fiecare cuvânt și compari. Lungimea îți spune ce algoritm să folosești.

```python
import hashlib


def sparge_hashuri():
    cale_hashuri = Path("probe/hashuri.txt")
    cale_cuvinte = Path("probe/wordlist.txt")

    hashuri = cale_hashuri.read_text(encoding="utf-8").splitlines()
    cuvinte = cale_cuvinte.read_text(encoding="latin1").splitlines()

    for tinta in hashuri:
        tinta = tinta.strip()

        for cuvant in cuvinte:
            rezultat = hashlib.md5(cuvant.encode("utf-8")).hexdigest()

            if rezultat == tinta:
                print("Hash găsit:", tinta)
                print("Parola:", cuvant)
                break


sparge_hashuri()
```

Fig.6.1-Codul sursă (decode.py)

```
Hash găsit: 5f4dcc3b5aa765d61d8327deb882cf99
Parola: password
Hash găsit: 8621ffdbc5698829397d97767ac13db3
Parola: dragon
Hash găsit: d8578edf8458ce06fbc5bb76a58c5ca4
Parola: qwerty
Hash găsit: 0d107d09f5bbe40cade3de5c71e9e9b7
Parola: letmein
Hash găsit: d0763edaa9d9bd2a9516280e9044d885
Parola: monkey
```

Fig.6.2-Rezultatul codului (decode.py)

Capitolul 7. argparse, fișa JSON și un test (16 min)

Leagă totul într-o unealtă comandată din afară, salvează rezultatul ca fișă, și scrie un test.

```
import argparse, json
p = argparse.ArgumentParser()
p.add_argument("intrare")          # sir sau cale
p.add_argument("--fisier", action="store_true")
a = p.parse_args()
# TODO: daca a.fisier, citeste octetii din cale;
#       altfel trateaza intrarea ca sir
# TODO: salveaza {strat, rezultat} in
#       arsenal/forensics/fisa.json
```

Un test scurt care ține decoderul cinstit:

```
# teste/test_decode.py
from arsenal.forensics.decode import desfa
def test_base64():
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"
```

```python
fisa = {
    "straturi": ["base64", "base64", "hex"],
    "rezultat": mesaj_clar.decode("utf-8")
}

cale_fisa = Path("arsenal/forensics/fisa.json")
cale_fisa.write_text(
    json.dumps(fisa, indent=2, ensure_ascii=False),
    encoding="utf-8"
)

print("Fișa salvată în:", cale_fisa)
parser = argparse.ArgumentParser()
parser.add_argument("intrare", nargs="?", help="Șirul de decodat sau calea fișierului")
parser.add_argument("--fisier", action="store_true", help="Intrarea este o cale de fișier")

argumente = parser.parse_args()

if argumente.intrare:
    if argumente.fisier:
        text_intrare = Path(argumente.intrare).read_text(
            encoding="utf-8"
        ).strip()
    else:
        text_intrare = argumente.intrare

    rezultat = desfa_straturi(text_intrare)
    print("Rezultat:", rezultat.decode("utf-8", errors="replace"))
```

Fig.7.1-Codul sursă (decode.py)

```
Fișa salvată în: arsenal\forensics\fisa.json
```

Fig.7.2-Rezultatul codului (decode.py)

```python
from arsenal.forensics.decode import desfa


def test_base64():
    strat, rezultat = desfa("U2FsdXQ=")

    assert strat == "base64"
    assert rezultat == b"Salut"
```

Fig.7.3-Codul sursă (test_decode.py)

```
(.venv) PS C:\Users\moroz\OneDrive\Documents\laborator6> .\.venv\Scripts\python.exe -m pytest teste\test_decode.py
============================= test session starts ==============================
platform win32 -- Python 3.15.0, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\moroz\OneDrive\Documents\laborator6
collected 1 item

teste\test_decode.py .                                                   [100%]

============================== 1 passed in 0.09s ===============================
(.venv) PS C:\Users\moroz\OneDrive\Documents\laborator6>
```

Fig.7.4-Rezultatul rulării (test_decode.py)



Fig.7.5- Codul fisa.json

Capitolul 8. Magic bytes: tipul real, nu extensia (12 min)

Deschide un fișier în mod binar și citește-i primii octeți. Compară-i cu semnăturile cunoscute.

```
def tip_real(cale):
    cap = open(cale, "rb").read(8)
    # TODO: daca cap incepe cu b"%PDF" -> "pdf"
    #       b"\x89PNG" -> "png" ; b"\xff\xd8" -> "jpeg"
    #       b"PK\x03\x04" -> "zip/docx"
    return cap.hex()
print(tip_real("probe/foto.jpg"))
```

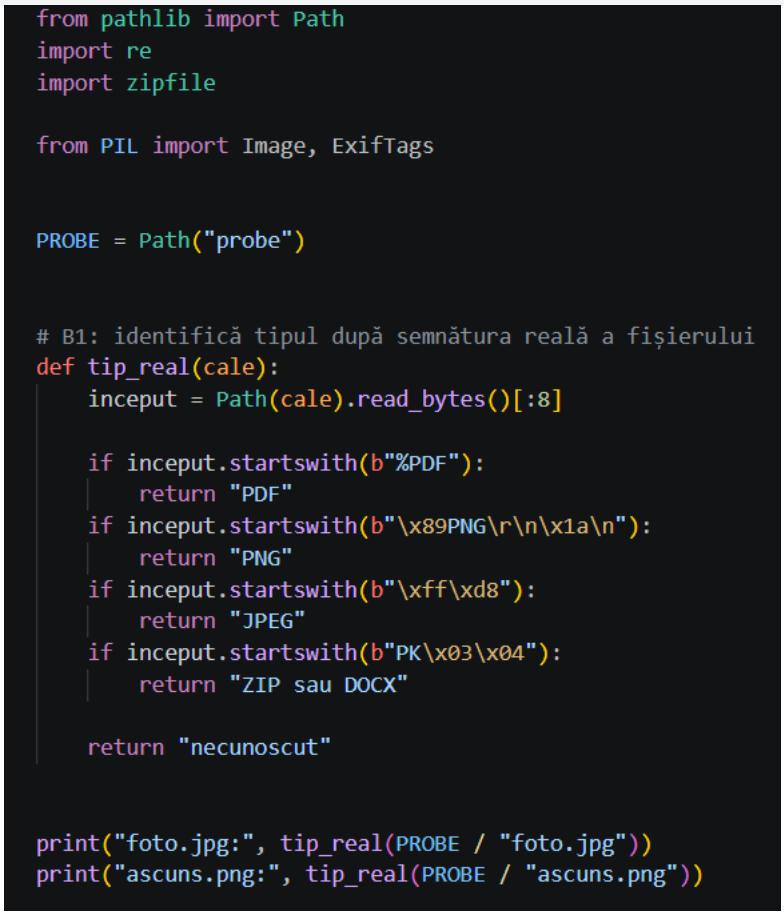


Fig.8.1-Codul sursă (forensics.py)

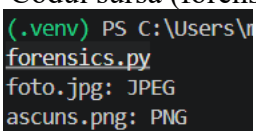


Fig.8.2-Rezultatul codului (forensics.py)

Capitolul 9. strings: textul citibil dintr-un binar (12 min)

Scoate secvențele de caractere imprimabile dintr-un fișier, cu un regex pe octeți.

```
import re
date = open("probe/ascuns.png", "rb").read()
# TODO: for m in re.findall(rb"[ -~]{4,}", date):
#           print(m.decode())
# cauta un indiciu: o cale, un mesaj, un nume de fisier
```

```
# B2: afișează secvențele de text imprimabil din fișier
date_png = (PROBE / "ascuns.png").read_bytes()
print("\nText imprimabil găsit în ascuns.png:")

for secventa in re.findall(rb"[ -~]{4,}", date_png):
    print(secventa.decode("ascii"))
```

Fig.9.1-Codul sursă (forensics.py)

```
Text imprimabil găsit în ascuns.png:
IHDR
IDATx
IEND
secret.txtFLAG{ascuns_prin_carving}
secret.txtPK
```

Capitolul 10. Metadate EXIF și GPS din poză (16 min)

Citește blocul EXIF din probe/foto.jpg și scoate coordonatele GPS. Convertește-le în grade zecimale.

```
from PIL import Image
from PIL.ExifTags import TAGS
exif = Image.open("probe/foto.jpg")._getexif() or {}
for id, val in exif.items():
    print(TAGS.get(id, id), "=", val)
# TODO: ia blocul GPS (tag 34853) si tipareste
#       latitudinea si longitudinea
```

```python
# B3: citește metadatele EXIF și coordonatele GPS
imagine = Image.open(PROBE / "foto.jpg")
exif = imagine.getexif()

print("\nMetadate EXIF:")
for id_tag, valoare in exif.items():
    nume = ExifTags.TAGS.get(id_tag, id_tag)
    if id_tag != 34853:
        print(nume, "=", valoare)

gps_raw = exif.get_ifd(34853)
gps = {
    ExifTags.GPSTAGS.get(id_tag, id_tag): valoare
    for id_tag, valoare in gps_raw.items()
}

print("\nMetadate GPS:", gps)


def in_grade_zecimale(coordonate, directie):
    grade, minute, secunde = [float(x) for x in coordonate]
    rezultat = grade + minute / 60 + secunde / 3600

    if directie in ("S", "W"):
        rezultat = -rezultat

    return rezultat


latitudine = in_grade_zecimale(
    gps["GPSLatitude"], gps["GPSLatitudeRef"]
)
longitudine = in_grade_zecimale(
    gps["GPSLongitude"], gps["GPSLongitudeRef"]
)

print("Latitudine:", latitudine)
print("Longitudine:", longitudine)
```

Fig.10.1-Codul sursă (forensics.py)

```
Metadate EXIF:
Model = iPhone 13
DateTime = 2026:09:14 10:30:00
Make = Apple

Metadate GPS: {'GPSLatitudeRef': 'N', 'GPSLatitude': (47.0, 1.0, 0.0), 'GPSLongitudeRef': 'E', 'GPSLongitude': (28.0, 51.0, 27.0)}
Latitudine: 47.016666666666666
Longitudine: 28.8575
```

Fig.10.2-Rezultatul codului (forensics.py)

Capitolul 11. File carving: scoți fișierul ascuns (14 min)

probe/ascuns.png are o arhivă ZIP lipită la coadă. Găsește-i semnătura și salveaz-o separat.

```
date = open("probe/ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")
# TODO: daca i != -1:
#   open("gasit.zip","wb").write(date[i:])
#   print("arhiva extrasa de la octetul", i)
# apoi dezarhiveaza si vezi ce era inauntru
```

```
# B4: găsește și extrage ZIP-ul lipit la sfârșitul imaginii PNG
pozitie = date_png.find(b"PK\x03\x04")

if pozitie == -1:
    print("\nNu am găsit arhiva ZIP.")
else:
    cale_zip = Path("gasit.zip")
    cale_zip.write_bytes(date_png[pozitie:])
    print("\nArhiva extrasă la octetul:", pozitie)

    with zipfile.ZipFile(cale_zip) as arhiva:
        print("Fișiere în arhivă:", arhiva.namelist())
        for nume in arhiva.namelist():
            continut = arhiva.read(nume).decode("utf-8")
            print(nume, "conține:", continut)
```

Fig.11.1-Codul sursă (forensics.py)

```
Arhiva extrasă la octetul: 782
Fișiere în arhivă: ['secret.txt']
secret.txt conține: FLAG{ascuns_prin_carving}
```

Fig.11.2-Rezultatul codului (forensics.py)

Capitolul 12. Ascunzi un mesaj prin LSB (16 min)

Scrie un mesaj în ultimii biți ai pixelilor dintr-o poză gazdă. Rezultatul arată identic cu originalul.

```
from PIL import Image
img = Image.open("probe/foto.jpg").convert("RGB")
px = list(img.getdata())
mesaj = b"FLAG{ascuns_in_pixeli}"
biti = "".join(f"{x:08b}" for x in mesaj)
# TODO: pentru fiecare bit, ia un canal si pune
#   canal = (canal & ~1) | int(bit)
# TODO: img.putdata(px_nou); img.save("stego.png")
```

```python
            # B5: ascunde mesajul în ultimii biți ai canalelor RGB
def ascunde_mesaj(cale_imagine, mesaj, cale_iesire):
    imagine = Image.open(cale_imagine).convert("RGB")
    pixeli = list(imagine.getdata())

    date_mesaj = mesaj.encode("utf-8") + b"\0"
    biti = "".join(f"{octet:08b}" for octet in date_mesaj)

    canale = [canal for pixel in pixeli for canal in pixel]

    if len(biti) > len(canale):
        raise ValueError("Mesajul este prea lung pentru imagine.")

    for i, bit in enumerate(biti):
        canale[i] = (canale[i] & 254) | int(bit)

    pixeli_noi = [
        tuple(canale[i:i + 3])
        for i in range(0, len(canale), 3)
    ]

    imagine.putdata(pixeli_noi)
    imagine.save(cale_iesire, format="PNG")
    print("\nImaginea cu mesajul ascuns a fost salvată:", cale_iesire)
```

Fig.12.1-Codul sursă (forensics.py)

```
c:\Users\moroz\OneDrive\Documents\laborator6\forensics.py:97: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_fla
ttened_data instead.
  pixeli = list(imagine.getdata())

Imaginea cu mesajul ascuns a fost salvată: stego.png
```

Fig.12.2-Rezultatul codului (forensics.py)

Capitolul 13. Dezvălui mesajul înapoi (14 min)

Citește ultimii biți ai pixelilor din stego.png, regrupează-i câte 8 și reconstituie mesajul.

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    # TODO: aduna ultimii biti (r&1, g&1, b&1)
    pass
# TODO: grupeaza cate 8 biti -> octeti -> text
#       opreste-te la primul octet 0 (final de mesaj)
```

```python
# B6: citește ultimii biți și reconstruiește mesajul
def dezvaluie_mesaj(cale_imagine):
    imagine = Image.open(cale_imagine).convert("RGB")
    biti = "".join(
        str(canal & 1)
        for pixel in imagine.getdata()
        for canal in pixel
    )

    rezultat = bytearray()

    for i in range(0, len(biti) - 7, 8):
        octet = int(biti[i:i + 8], 2)

        if octet == 0:
            break

        rezultat.append(octet)

    return rezultat.decode("utf-8")


cale_stego = Path("stego.png")
ascunde_mesaj(
    PROBE / "foto.jpg",
    "FLAG{ascuns_in_pixeli}",
    cale_stego
)

mesaj_extras = dezvaluie_mesaj(cale_stego)
print("Mesajul dezvăluit:", mesaj_extras)
```

Fig.13.1-Codul sursă (forensics.py)

```
  for pixel in imagine.getdata()
Mesajul dezvăluit: FLAG{ascuns_in_pixeli}
```

Fig.13.2-Rezultatul codului (forensics.py)

CONCLUZII

În această lucrare am decodat mesaje, am spart XOR și hashuri, am analizat metadate și am extras fișiere ascunse. Am aplicat și metoda LSB pentru a ascunde și recupera un mesaj dintr-o imagine.

BIBLIOGRAFIE

1. Visual Studio Code